GIT PARA INICIANTES

Um guia prático para trabalhar em grupo sem perder o código dos colegas

O que você vai aprender

- O que é Git e para que serve
- O que é um repositório e como clonar um projeto
- Como usar git add, git commit, git push e git pull
- Como colaborar com o grupo da faculdade com mais segurança

Exemplo usado neste guia: um grupo de faculdade criando o site de uma administradora de condomínios. Uma pessoa faz a página inicial, outra trabalha em Serviços e outra cuida de Contato. Todos colaboram no mesmo projeto, sem precisar trocar arquivos por WhatsApp.

Pré-requisitos: ter uma conta no GitHub, o Git instalado no computador e acesso ao repositório do grupo. Os comandos abaixo são executados no terminal (por exemplo, Git Bash, PowerShell ou terminal do VS Code).

Importante: Git e GitHub não são a mesma coisa. Git é a ferramenta que controla as versões do código; GitHub é um serviço online onde o repositório pode ficar hospedado e ser compartilhado.

1. O que é Git? Imagine um trabalho em grupo

Pense no site da faculdade como um trabalho de várias pessoas. Sem controle de versão, alguém pode enviar um arquivo chamado index-final-agora-vai-2.html, outra pessoa sobrescreve o CSS e ninguém sabe quem mudou o quê. O Git ajuda a registrar as mudanças e permite consultar versões anteriores.

Uma analogia simples

• Git: como um histórico organizado do projeto, com registros de quem mudou o quê e por quê.

• Repositório (repo): a pasta do projeto acompanhada pelo Git, incluindo o histórico de alterações.

• Commit: um registro/snapshot de um conjunto de mudanças, com uma mensagem explicando o que foi feito.

• GitHub: como uma estante compartilhada na internet onde o grupo guarda o repositório.

• Push: enviar seus commits locais para o repositório remoto.

• Pull: trazer alterações do repositório remoto para a sua cópia local, integrando-as.

O Git não salva automaticamente tudo que você digita. Você escolhe quais mudanças entram no próximo commit e quando registrar esse ponto do histórico.

2. O que é um repositório?

É o projeto com seus arquivos e o histórico de versões. Em um site, pode conter HTML, CSS, JavaScript, imagens e documentação.

Repositório remoto e local

• Remoto: a versão hospedada no GitHub, acessível ao grupo.

• Local: a cópia que está no seu computador, onde você edita e testa o código.

Exemplo: o repositório remoto é a versão compartilhada do site. Cada integrante tem uma cópia local para trabalhar. Depois, cada um envia suas alterações para o remoto.

Antes de começar: confira se o Git está instalado

```
git --version
```

Se aparecer uma versão, como `git version 2.x.x`, o Git está instalado. Se o comando não for reconhecido, instale o Git para Windows e abra um novo terminal.

Configure seu nome e e-mail (uma vez por computador/usuário)

```
git config --global user.name "Seu Nome"
git config --global user.email "seu-email@example.com"
```

Use o nome e o e-mail que deseja associar aos commits. O `--global` vale para seus repositórios no usuário atual do computador. Em computadores compartilhados, avalie configurar por projeto sem `--global`.

3. git clone — baixar o projeto para seu computador

Quando usar: na primeira vez que você vai trabalhar em um repositório que já existe no GitHub.

Passo a passo

1. Abra a página do repositório no GitHub.
2. Clique em Code e copie a URL HTTPS.
3. No terminal, entre na pasta onde quer guardar o projeto. Exemplo: a pasta de projetos da

faculdade.

```
cd "C:\Users\SeuUsuario\Documents\Faculdade"
```

4. Execute o clone usando a URL copiada do seu próprio repositório:

```
git clone https://github.com/ORGANIZACAO/NOME-DO-REPOSITORIO.git
```

5. Entre na pasta criada pelo clone:

```
cd NOME-DO-REPOSITORIO
```

O que aconteceu? O Git criou uma cópia local do projeto e trouxe o histórico de versões. Você normalmente faz `git clone` uma vez por cópia de trabalho; depois usa `git pull` para atualizar essa cópia.

Atenção: substitua a URL de exemplo pela URL real do repositório do seu grupo. Se o repositório for privado, você precisa ter permissão de acesso.

4. git status — confira antes de agir

Este comando não altera arquivos. Ele mostra em qual branch você está e quais arquivos foram modificados, preparados ou ainda não são acompanhados pelo Git.

```
git status
```

Exemplo: se você editar `index.html`, o Git pode mostrar que o arquivo foi modificado. É um bom hábito executar `git status` antes de adicionar ou enviar mudanças.

5. git add — escolher o que vai entrar no próximo registro

Quando usar: depois de editar arquivos e antes de criar um commit.

Imagine que você alterou o título da Home e também começou a mexer em uma página que ainda não está pronta. Com `git add`, você escolhe quais mudanças quer incluir no próximo commit.

Adicionar um arquivo específico

```
git add index.html
```

Adicionar todos os arquivos modificados e novos da pasta atual

```
git add .
```

O ponto significa “a partir daqui, adicionar as mudanças encontradas nesta pasta e nas subpastas”. Confira o resultado com `git status`.

Importante: `git add` não envia nada ao GitHub. Ele prepara mudanças na sua cópia local para o próximo commit.

6. git commit — registrar uma versão

Quando usar: quando você preparou um conjunto coerente de alterações e quer registrar o que fez.

```
git commit -m "feat: cria estrutura inicial da página inicial"
```

A opção `-m` permite escrever a mensagem do commit no próprio comando. A mensagem deve explicar a mudança, não apenas dizer “alterações” ou “teste”.

Exemplos de mensagens úteis

- `feat: adiciona seção de serviços` — funcionalidade ou conteúdo novo.
- `fix: corrige espaçamento do cabeçalho` — correção.
- `docs: atualiza instruções do projeto` — documentação.

Um commit fica registrado no histórico local. Ele ainda não aparece no GitHub até você fazer o push.

7. git push — enviar seus commits ao GitHub

Quando usar: depois de criar um commit, para publicar seus commits no repositório remoto.

```
git push
```

Em alguns projetos, na primeira publicação de uma branch nova, pode ser necessário definir o destino:

```
git push -u origin nome-da-branch
```

Exemplo: você terminou a seção de serviços, fez add e commit e agora usa push. Seus commits são enviados para o GitHub, desde que você tenha permissão e que não exista um conflito que impeça a

atualização.

> Push não é o mesmo que salvar: o commit já fica salvo no histórico local; o push publica esse registro no remoto.

8. git pull — trazer as mudanças do grupo

Quando usar: antes de começar a trabalhar e sempre que precisar atualizar sua cópia com o que os colegas publicaram.

```
git pull
```

Exemplo real: você vai trabalhar na página Contato. Enquanto isso, uma colega atualizou o cabeçalho compartilhado e enviou as mudanças ao GitHub. Antes de continuar, você faz `git pull` para integrar as mudanças à sua cópia local.

> O `git pull` busca as mudanças remotas e tenta integrá-las à branch atual. Se você e outra pessoa editaram a mesma parte de um arquivo, pode ocorrer um conflito que precisa ser resolvido.

Regra prática: antes de começar uma tarefa, verifique o estado do projeto e atualize sua branch. Se você já tem alterações locais, não faça pull às cegas: confira com `git status` e, se necessário, registre ou guarde suas mudanças primeiro.

9. Fluxo completo: da edição ao GitHub

Imagine que você ficou responsável pelo título e pela apresentação da Home do site da faculdade.

```
# 1. Veja como está sua cópia
git status

# 2. Atualize com o trabalho já publicado pelos colegas
git pull

# 3. Edite os arquivos no VS Code

# 4. Escolha as mudanças que entrarão no commit
git add index.html css/style.css

# 5. Registre o que você fez
git commit -m "feat: atualiza apresentação da home"

# 6. Envie o commit ao GitHub
git push
```

Esse é o ciclo básico: atualizar → editar → add → commit → push. O nome do arquivo e a mensagem são exemplos; use os arquivos e a descrição correspondentes à sua tarefa.

10. Como evitar problemas trabalhando em grupo

Combine responsabilidades

• Dividam as páginas e componentes: por exemplo, Home, Serviços e Contato.

• Evitem editar o mesmo arquivo ao mesmo tempo sem combinar, principalmente se o grupo ainda está aprendendo.

• Antes de mudar arquivos compartilhados, avisem no canal do grupo.

Boas práticas

• Façam commits pequenos e com uma mensagem clara.

• Executem `git status` antes de add, commit e push.

• Façam pull antes de iniciar uma tarefa, desde que suas alterações locais estejam em uma situação segura.

• Não enviem senhas, tokens, chaves privadas ou arquivos de configuração com segredos.

• Não usem `git push --force` como tentativa de resolver qualquer erro. Ele pode sobrescrever histórico remoto.

• Se aparecer conflito, parem e peçam ajuda ao grupo antes de apagar ou aceitar mudanças sem entender.

Se o push for rejeitado

Uma causa comum é que o GitHub tem commits que você ainda não tem localmente. Leia a mensagem de erro, confira o status e, quando apropriado, faça pull para integrar as mudanças. Se houver conflito, resolva-o antes de tentar enviar novamente. Outro motivo pode ser falta de permissão no repositório.

11. Erros comuns traduzidos

Mensagem/situação	O que significa / próximo passo
fatal: not a git repository	Você não está dentro da pasta do repositório. Use `cd` para entrar na pasta correta e confira se ela contém a pasta `.git`.
nothing to commit	Não há mudanças preparadas para registrar. Confira `git status`; talvez nada tenha mudado ou você ainda não tenha feito add.
rejected / fetch first	O remoto pode ter commits que faltam na sua cópia. Confira a mensagem, atualize e integre as mudanças antes de tentar o push novamente.
Permission denied / 403	Sua conta pode não ter permissão, ou o terminal pode estar autenticado com outra conta. Peça acesso ou corrija a autenticação.
CONFLICT	Duas mudanças não puderam ser integradas automaticamente. Abra os arquivos marcados, compare as versões e combine com o colega qual resultado manter.

12. Exercício prático para o grupo

Façam este exercício em um repositório de teste ou em uma branch combinada com o grupo. Não usem a branch principal de um projeto importante para aprender.

Desafio: cada pessoa vai adicionar uma linha à seção “Equipe” em um arquivo de teste chamado equipe.txt.

```
# Crie o arquivo equipe.txt no editor e escreva seu nome e sua tarefa.
# Depois, no terminal:
git status
git add equipe.txt
git commit -m "docs: adiciona integrante à equipe"
git push
```

Depois que um colega publicar a alteração, a outra pessoa executa `git pull` e verifica se a mudança chegou à sua cópia. Se duas pessoas editarem a mesma linha, usem isso como demonstração de conflito — com orientação — e pratiquem a resolução.

Checklist para lembrar

• Estou na pasta certa? → git status

• Preciso trazer o trabalho publicado pelos colegas? → git pull

• Quais arquivos quero incluir? → git add arquivo

• Quero registrar as mudanças? → git commit -m "mensagem clara"

• Quero publicar meus commits? → git push

Resumo final: clone é para obter uma cópia inicial; pull atualiza uma cópia existente; add seleciona mudanças; commit registra as mudanças localmente; push envia os commits ao remoto.